Frozen self-supervised features and temporal context for free-living activity recognition on CAPTURE-24

Alex Ronczewski
alex.ronczewski@gmail.com

Abstract

CAPTURE-24 is a public dataset in which 151 people wore a wrist accelerometer for about a day while a camera photographed what they were doing. The task is to label each 10-second window of the signal with one of 10 activities. I rebuilt the dataset paper's baseline, a random forest smoothed by a hidden Markov model (HMM), and scored Cohen's κ 0.66 against its published 0.64. I then took a network that other researchers had pretrained on 700,000 days of unlabelled wrist data, kept it frozen, and fed its features into a small recurrent network that reads about ten minutes of signal at a time. This reached κ 0.79 on 51 held-out participants, 0.10 above my baseline. Against the same pretrained network followed by an HMM, the gain is 0.04 κ with no change in macro-F1. Part of that gain comes from the rule that turns output probabilities into labels.

1 Introduction

Large health studies put accelerometers on people's wrists to measure how long they sleep, sit and move. Turning the signal into activities requires labelled examples from everyday life. CAPTURE-24 (Chan et al., 2024) is one such dataset. Each participant wore a wrist accelerometer and a camera for about a day. The camera took a photo every 20 to 40 seconds, and annotators labelled the activity from the photos. The input to a model is three-axis acceleration sampled 100 times per second, and the output is one of 10 activities for each 10-second window. The dataset paper reports a random forest and a convolutional neural network (CNN), each followed by an HMM, and tests them on participants P101 to P151, who are never used for training.

Activities such as cooking, a commute or an afternoon at a desk last minutes, so a model that sees 10 seconds at a time misses most of the context. Some classes, such as household chores and manual work, also look almost identical from the wrist. I tried letting the model see about ten minutes of signal at once, and starting from a network pretrained on a much larger dataset (Yuan et al., 2024). Before trying either, I checked that I could reproduce the classical baseline and that my evaluation was sound.

Below I cover the baseline and two evaluation mistakes I found while rebuilding it, a quirk of the labels that affects every score on this dataset, the models I tried, and how much each idea contributed. Figure 1 shows the baseline and my best model on six hours of one held-out participant.

2 Related work

Willetts et al. (2018) introduced the random forest + HMM approach for wrist data in UK Biobank, along with the 10 activity classes I use. Chan et al. (2024) released CAPTURE-24 with random forest and CNN baselines and named self-supervised learning, where a network learns from unlabelled data, as future work. Yuan et al. (2024), from the same Oxford group, then pretrained a network this way on 700,000 days of unlabelled UK Biobank wrist data (Doherty et al., 2017). They tested it on CAPTURE-24, but only on a simpler 4-class version of the task. Their tool ActiNet (Acquah et al., 2025) uses the same network, followed by an HMM, to sort 30-second windows into those 4 classes. The first version of the ActiNet paper froze the pretrained network and trained only a final layer. The current version fine-tunes the whole network, and its authors list recurrent models as future work. Haresamudram et al. (2022) compared self-supervised



Figure 1: Six hours of held-out participant P101: camera labels, the random forest + HMM (RF-HMM) and my best model (SSL-BiGRU). Only windows with a camera label are drawn.

methods for activity recognition across datasets, and Ordóñez and Roggen (2016) combined convolutional and recurrent layers to give activity models temporal context. I found no published 10-class result for this pretrained network on CAPTURE-24, and the 4-class numbers cannot be compared with mine.

3 Data and protocol

Each recording (Chan Chang et al., 2021) holds three-axis acceleration in units of g, sampled at 100 Hz, with a label for every sample. The 10 activities are sleep, sitting, standing, household chores, manual work, walking, mixed activity, vehicle, sports and bicycling. I cut the recordings into non-overlapping 10-second windows, discarded incomplete windows, and gave each window the most common label among its samples. Windows that were mostly unlabelled stayed unlabelled. Models saw them as input, but I never trained or scored on them. This gives 922,199 labelled windows, the same count as the dataset paper.

I made every design decision with 5-fold cross-validation on participants P001 to P100. Each person's data sits in one fold, so models are always tested on people they have not seen. I then trained each final model on all of P001 to P100 and scored it on P101 to P151, the held-out participants.

I report two metrics. Cohen's κ (Cohen, 1960) measures agreement between predictions and labels after correcting for the agreement expected by chance. Common activities such as sleep and sitting dominate it. Macro-F1 is the unweighted mean of the per-activity F1 scores, where F1 is the harmonic mean of precision and recall. A rare activity like sports counts as much as sleep. The code uses scikit-learn (Pedregosa et al., 2011), imbalanced-learn (Lemaître et al., 2017) and PyTorch (Paszke et al., 2019).

To estimate uncertainty, I bootstrap the held-out participants: I resample the 51 people with replacement 2,000 times and report the middle 95% of scores as an interval (Field and Welsh, 2007). To compare two models, I compute their difference within each resample, which gives a paired interval. Over the project I scored models on the held-out participants six times, and the intervals do not correct for that.

3.1 Label gaps

Annotators almost always left a short unlabelled gap where one activity changes to another. Of 609,580 pairs of neighbouring labelled windows, only 46 have two different activities. Almost every label change goes into or out of "unlabelled". An HMM that learns transition rates from these labels therefore concludes that people almost never switch activity. It also means every scored window sits inside a bout, a continuous stretch of one activity, so no score on this dataset measures how a model handles a change of activity.

4 Reproduction and pitfalls

My random forest (RF) used 300 trees on 39 hand-crafted features per window. Thirty describe the acceleration magnitude: moments, quantiles, autocorrelation, and spectral and peak features. Nine describe orientation: per-axis means and standard deviations, and correlations between axes. I used imbalanced-learn's balanced random forest (Chen et al., 2004), in which each tree trains on an equal number of windows from every activity, so rare activities are not swamped.

Table 1: Reproduction. Cross-validation means on P001 to P100 and held-out scores on P101 to P151, with 95% intervals in brackets. The last row is RF-HMM minus Paper RF-HMM, paired on the held-out participants.

	Cross-validation		Held out	
Model	Macro-F1	κ	Macro-F1	κ
Paper RF-HMM, published			0.486	0.637
Paper RF-HMM, reimplemented	0.517	0.670	0.535 [0.490, 0.566]	0.660 [0.630, 0.692]
RF	0.406	0.535	0.415 [0.380, 0.438]	0.530 [0.503, 0.555]
RF-HMM	0.522	0.708	0.566 [0.524, 0.595]	0.690 [0.653, 0.725]
RF-HMM − Paper RF-HMM			+0.031 [−0.011, 0.080]	+0.029 [0.001, 0.058]

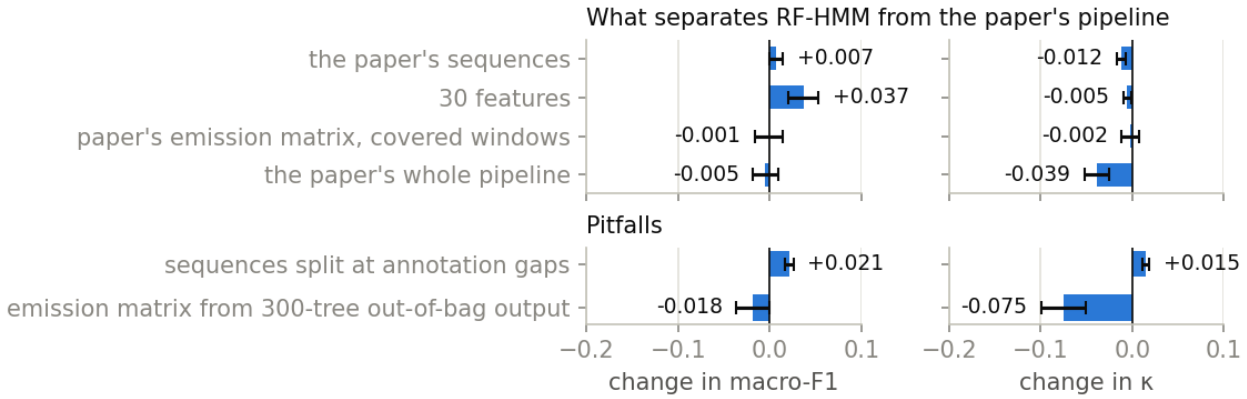


Figure 2: Change in RF-HMM's cross-validation macro-F1 and κ when one choice is swapped: mean over the five folds, with standard-error bars. Top: the paper's choices. Bottom: the two pitfalls.

A random forest judges each window on its own, so its predictions flicker from window to window. RF-HMM fixes this with an HMM (Rabiner, 1989). The HMM's starting probabilities, and its probabilities of moving from one activity to another, are counted from the training participants. Because each tree saw balanced classes, I treat the forest's output as the likelihood of the window under each activity. The Viterbi algorithm then finds the most likely sequence of activities for the whole recording. In effect, the HMM holds the current activity until the forest's evidence against it is strong and lasts long enough. RF-HMM decodes every window, labelled or not, because a deployed model would not know where the labels are.

I also reimplemented the dataset paper's pipeline (Paper RF-HMM) from its published code, fixing every setting before looking at any score. It differs from mine in its tree count (3,000), its features (only the 30 magnitude ones), how it feeds the forest's output to the HMM, and which windows it decodes. My reimplementation scored slightly above the published numbers, and the published κ falls inside its interval (Table 1). My RF-HMM scored higher still, so I used it as the baseline from here on.

To find which of the paper's choices made the difference, I swapped them into my RF-HMM one at a time in cross-validation. Each changed κ by at most 0.012 (Figure 2). The paper's whole pipeline was 0.039 below mine.

I also measured two evaluation mistakes. The first is a label leak. If you split each recording into sequences wherever the labels stop, the gaps tell the HMM where each activity starts and ends. That raised macro-F1 by about 0.02 in cross-validation, but a deployed model never has that information. The paper's pipeline decodes only labelled windows. That is a milder form of the same leak, and it did not help in my tests.

The second comes from how imbalanced-learn computes out-of-bag predictions. The paper builds its HMM from out-of-bag predictions, where each training window is scored only by trees that did not train on it. In a balanced forest, a window is out-of-bag for a tree only if it was drawn into that tree's class-balanced sample and then missed by the tree's bootstrap. Each tree draws only as many windows of each activity as the rarest activity has, so a given sleep or sitting window is out-of-bag for only about one tree in 300. With 300 trees, 29% of sleep and sitting windows received no out-of-bag prediction. The library fills them with

Table 2: The models. SSL models use the pretrained (self-supervised) encoder; Feat models use the hand-crafted features.

Model	Input to the classifier	Context from neighbouring windows
RF-HMM	39 hand-crafted features, random forest	HMM
SSL-FT-HMM	raw signal, encoder fine-tuned	HMM
SSL-HMM	frozen encoder features	HMM
SSL-BiGRU	frozen encoder features	BiGRU over the recording
Feat-HMM	39 hand-crafted features	HMM
Feat-BiGRU	39 hand-crafted features	BiGRU over the recording

zeros and prints only a warning. Building the HMM from that output cost about 0.08 κ (Figure 2). The paper's 3,000 trees avoid the problem, and so does dropping the uncovered windows.

5 Models

The encoder is `harnet10` from `ssl-wearables` (Yuan et al., 2024; OxWearables, 2024), an 18-layer 1D ResNet (a convolutional network with skip connections). Its authors pretrained it on unlabelled UK Biobank data, with no CAPTURE-24 data. It reads 10 seconds of signal at 30 Hz, so I resampled each window from 1,000 to 300 samples. For each window it outputs 1,024 features.

Table 2 lists the models. SSL-FT-HMM fine-tunes the encoder. I added a linear output layer for the 10 classes and retrained all layers on CAPTURE-24 with the encoder authors' settings: Adam with AMSGrad, learning rate 10^{-4} and batches of 1,000 windows. Training stopped when the loss on a fifth of the training participants, set aside for validation, stopped improving. An HMM then smooths the output, as in RF-HMM. This follows the current version of ActiNet.

SSL-BiGRU keeps the encoder frozen, so its weights never change. I ran every window through it once and fed the features of consecutive windows into a bidirectional GRU (Cho et al., 2014), a recurrent network that reads the sequence both forwards and backwards. It has 2 layers, 128 units per direction and dropout 0.5. I trained it on stretches of 64 windows, about ten minutes. Unlabelled windows were part of the input but contributed no loss. At test time it reads each whole recording.

As a control, SSL-HMM feeds the same frozen features to the same network one window at a time, then smooths the output with the HMM. This follows the first version of ActiNet, and its only difference from SSL-BiGRU is context. Feat-HMM and Feat-BiGRU repeat the two networks on the 39 hand-crafted features instead of the encoder's features. Comparing all four separates the effect of the encoder from the effect of context.

I fixed each model's settings, and the rule for choosing between candidates, before seeing any of its scores. I trained each network three times per fold with different random seeds, and each final model once on P001 to P100.

The decision rule turns a model's output probabilities into one predicted activity. A network trained on the data as-is learns that sleep and sitting are common and favours them. Dividing each probability by how often that activity occurs in training removes this bias and shifts predictions towards rare activities (Saerens et al., 2002). A parameter α sets the strength of the correction: each probability is divided by its activity's training frequency raised to the power α, and the largest result wins. At $\alpha = 0$ the bias is kept, and at $\alpha = 1$ it is fully removed. I scored every model at $\alpha \in \{0, 0.25, 0.5, 0.75, 1\}$. For the held-out scores I fixed α in advance. The HMM models use $\alpha = 1$, because the HMM expects the bias removed. For SSL-BiGRU, I chose between taking its top class ($\alpha = 0$) and passing its bias-removed output through the HMM. My rule was to pick the higher cross-validation macro-F1, with κ deciding if the two were within 0.005. They were, and the top class won on κ.

Table 3: Main results. Cross-validation means (networks averaged over three seeds) and held-out scores with 95% intervals. The bottom rows are paired differences on the held-out participants. I have only the published scores for the CNN + HMM, so it has no interval or paired difference.

	Cross-validation		Held out	
Model	Macro-F1	κ	Macro-F1	κ
CNN + HMM, published			0.576	0.735
RF-HMM	0.522	0.708	0.566 [0.524, 0.595]	0.690 [0.653, 0.725]
SSL-FT-HMM	0.578	0.741	0.619 [0.557, 0.659]	0.742 [0.702, 0.778]
SSL-HMM	0.603	0.776	0.638 [0.571, 0.681]	0.750 [0.717, 0.783]
SSL-BiGRU	0.580	0.814	0.642 [0.577, 0.677]	0.790 [0.758, 0.819]
SSL-BiGRU − RF-HMM			+0.076 [0.021, 0.111]	+0.100 [0.076, 0.124]
SSL-BiGRU − SSL-FT-HMM			+0.022 [−0.025, 0.057]	+0.048 [0.026, 0.071]
SSL-BiGRU − SSL-HMM			+0.004 [−0.025, 0.026]	+0.040 [0.023, 0.057]

6 Results and decomposition

SSL-BiGRU reached κ 0.79 on the held-out participants, 0.10 above RF-HMM (Table 3). Against SSL-HMM, which differs only in context, it gained 0.04 κ and scored higher for 42 of the 51 held-out participants. Their macro-F1 was level. The published CNN + HMM falls below SSL-BiGRU's κ interval. That CNN was trained from scratch, so part of the gap is the benefit of pretraining.

Most of RF-HMM's errors were far from activity changes. Windows within 30 seconds of a label change are only 8% of the data, and dropping them raised RF-HMM's cross-validation κ from just 0.71 to 0.74. Two thirds of its errors were more than a minute from any change. That suggests whole stretches of a few minutes that a 10-second classifier misreads. On windows 1 to 5 minutes from a change, context lowered the error rate from 0.25 for SSL-HMM to 0.21 for SSL-BiGRU. Adding an HMM after SSL-BiGRU lowered its cross-validation κ from 0.81 to 0.77, which suggests the network had already removed the flicker and the HMM only held activities too long.

The frozen encoder did at least as well as the fine-tuned one, in cross-validation and on the held-out participants (Table 3). The two setups also differ in their final layers, and I tried only one fine-tuning recipe, so a better recipe might still make fine-tuning worthwhile.

SSL-BiGRU keeps the bias towards common activities, which helps κ, and SSL-HMM removes it, which helps macro-F1 (Figure 3). With each model at its best α, SSL-BiGRU's cross-validation lead over SSL-HMM shrinks from 0.04 to 0.02 κ, and its lead over RF-HMM from 0.11 to 0.06. The same comparison across the four networks shows that the encoder and context each help less when the other is present. The encoder adds 0.07 κ without context and 0.03 with it. Context adds 0.06 on hand-crafted features and 0.02 on encoder features. No model passed a cross-validation macro-F1 of 0.60 at any α.

7 Discussion and limitations

A frozen pretrained encoder followed by a recurrent network over about ten minutes of windows gave the best κ I measured on the 10-class task. Against the other two encoder models the gain is in κ only, since both macro-F1 intervals include zero. Manual work and mixed activity stayed at or below F1 0.3 in cross-validation.

The results have several limitations:

- The held-out comparisons use different decision rules for different models.
- I chose models on the same cross-validation folds I report. I scored only one training run of each model on the held-out participants, and I used them six times over the project.
- All the data come from one group of UK adults wearing one type of device.
- Rare activities rest on few people: only 8 of the 51 held-out participants did any sport.

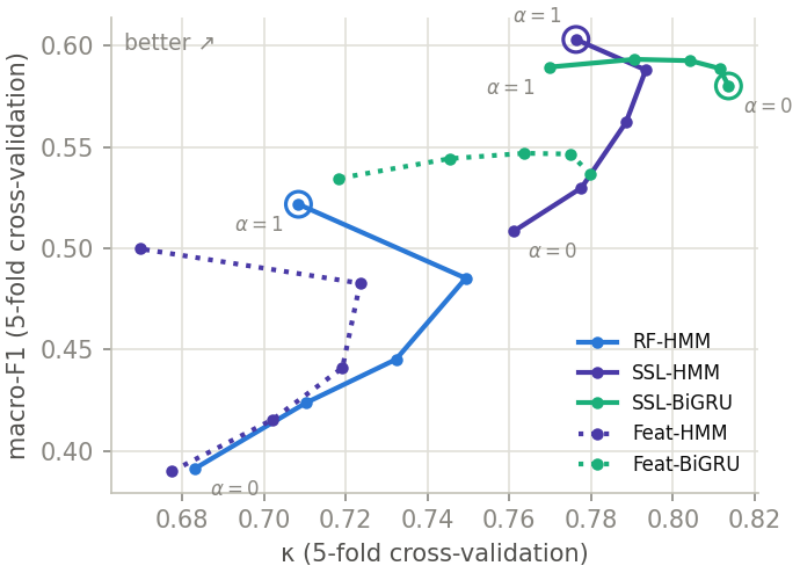


Figure 3: Cross-validation macro-F1 against κ as α goes from 0 (bias towards common activities kept) to 1 (bias removed). Rings mark the α used for the held-out scores. Dotted lines are the networks trained on hand-crafted features.

- The encoder's authors also built the dataset, so its design may suit this benchmark.
- SSL-BiGRU needs the whole recording before it labels anything, so it works only offline.

The next step is data labelled continuously through changes of activity, which would show how much each score depends on the label gaps. Until then, a frozen pretrained encoder with minutes of context is a strong default for this task, and anyone comparing models on it should report their decision rule. The code, notebooks and every number in this report are at `https://github.com/Alexr951/Capture-24`.

References

Aidan Acquah, Shing Chan, and Aiden Doherty. ActiNet: An open-source tool for activity intensity classification of wrist-worn accelerometry using self-supervised deep learning, 2025. arXiv:2510.01712, version 1 (October 2025) and version 2 (April 2026).

Shing Chan, Hang Yuan, Catherine Tong, Aidan Acquah, Abram Schonfeldt, Jonathan Gershuny, and Aiden Doherty. CAPTURE-24: A large dataset of wrist-worn activity tracker data collected in the wild for human activity recognition. *Scientific Data*, 11(1):1135, 2024. doi: 10.1038/s41597-024-03960-3.

S. Chan Chang, R. Walmsley, J. Gershuny, T. Harms, E. Thomas, K. Milton, P. Kelly, C. Foster, A. Wong, N. Gray, S. Haque, S. Hollowell, and A. Doherty. Capture-24: Activity tracker dataset for human activity recognition. University of Oxford, 2021.

Chao Chen, Andy Liaw, and Leo Breiman. Using random forest to learn imbalanced data. Technical Report 666, Department of Statistics, University of California, Berkeley, 2004.

Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder-decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, pages 1724–1734, 2014. doi: 10.3115/v1/D14-1179.

Jacob Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20 (1):37–46, 1960. doi: 10.1177/001316446002000104.

Aiden Doherty, Dan Jackson, Nils Hammerla, Thomas Plötz, Patrick Olivier, Malcolm H. Granat, Tom White, Vincent T. van Hees, Michael I. Trenell, Christopher G. Owen, Stephen J. Preece, Rob Gillions, Simon Sheard, Tim Peakman, Soren Brage, and Nicholas J. Wareham. Large scale population assessment of physical activity using wrist worn accelerometers: The UK Biobank study. *PLOS ONE*, 12(2):e0169649, 2017. doi: 10.1371/journal.pone.0169649.

C. A. Field and A. H. Welsh. Bootstrapping clustered data. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 69(3):369–390, 2007. doi: 10.1111/j.1467-9868.2007.00593.x.

Harish Haresamudram, Irfan Essa, and Thomas Plötz. Assessing the state of self-supervised human activity recognition using wearables. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, 6(3):1–47, 2022. doi: 10.1145/3550299.

Guillaume Lemaître, Fernando Nogueira, and Christos K. Aridas. Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning. *Journal of Machine Learning Research*, 18 (17):1–5, 2017.

Francisco Javier Ordóñez and Daniel Roggen. Deep convolutional and LSTM recurrent neural networks for multimodal wearable activity recognition. *Sensors*, 16(1):115, 2016. doi: 10.3390/s16010115.

OxWearables. ssl-wearables: Self-supervised learning for wearables using the UK Biobank. `https://github.com/OxWearables/ssl-wearables`, 2024. Commit 150550e.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, 2019.

Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

Lawrence R. Rabiner. A tutorial on hidden Markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2):257–286, 1989. doi: 10.1109/5.18626.

Marco Saerens, Patrice Latinne, and Christine Decaestecker. Adjusting the outputs of a classifier to new a priori probabilities: A simple procedure. *Neural Computation*, 14(1):21–41, 2002. doi: 10.1162/089976602753284446.

Matthew Willetts, Sven Hollowell, Louis Aslett, Chris Holmes, and Aiden Doherty. Statistical machine learning of sleep and physical activity phenotypes from sensor data in 96,220 UK Biobank participants. *Scientific Reports*, 8:7961, 2018. doi: 10.1038/s41598-018-26174-1.

Hang Yuan, Shing Chan, Andrew P. Creagh, Catherine Tong, Aidan Acquah, David A. Clifton, and Aiden Doherty. Self-supervised learning for human activity recognition using 700,000 person-days of wearable data. *npj Digital Medicine*, 7:91, 2024. doi: 10.1038/s41746-024-01062-3.